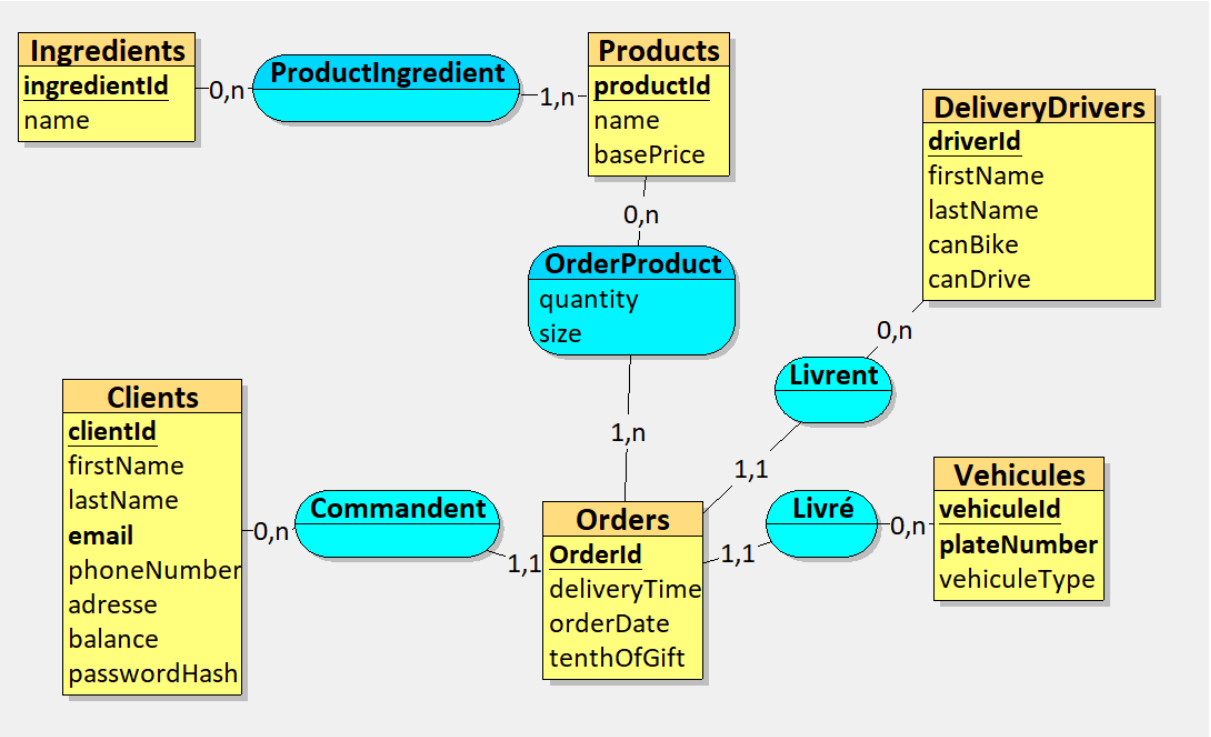
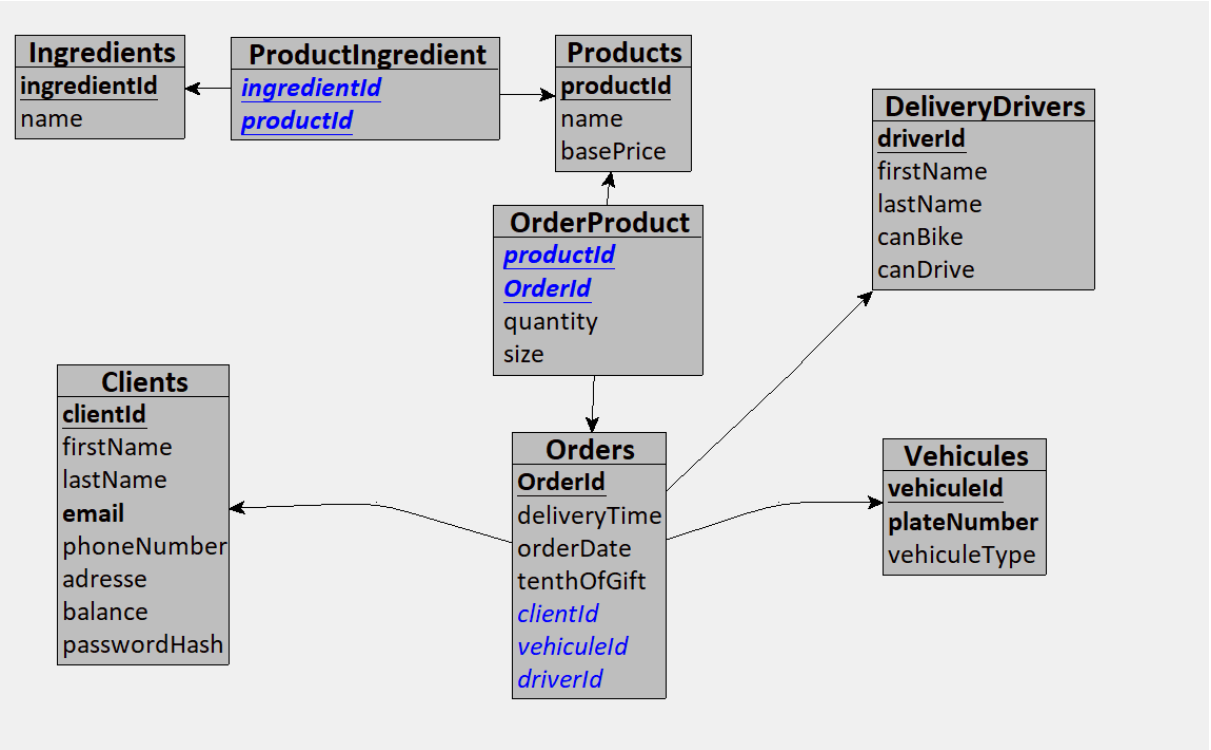
Projet Rapizz

Conception de la base de données

Elaboration du modèle entité-association



Modèle relationnel



Script de création des tables

```
create table Ingredients(
    id int primary key not null auto_increment,
    name varchar(255) not null,
    unique key uq_ingredients_name (name)
);

create table Products(
    id int primary key not null auto_increment,
    name varchar(255) not null,
    basePrice decimal(5,2) not null,
    key idx_products_name (name)
);

create table ProductIngredient(
    ingredientId int not null,
    productId int not null,
    primary key (ingredientId, productId),
    key idx_productingredient_productId (productId),
    constraint fk_productingredient_ingredient
        foreign key (ingredientId) references Ingredients(id) on delete cascade,
    constraint fk_productingredient_product
        foreign key (productId) references Products(id) on delete cascade
);

create table Clients(
    id int primary key not null auto_increment,
    email varchar(320) not null,
    phoneNumber varchar(32),
    firstName varchar(100),
    lastName varchar(100),
```

```
    address varchar(500),
    password_hash varchar(255) not null,
    balance DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    unique key uq_clients_email (email),
    key idx_clients_phoneNumber (phoneNumber)
);

create table Drivers(
    id int primary key not null auto_increment,
    firstName varchar(100),
    lastName varchar(100),
    canBike boolean not null default false,
    canDrive boolean not null default false
);

create table Vehicles(
    id int primary key not null auto_increment,
    plateNumber varchar(32) not null,
    brand varchar(100),
    model varchar(100),
    type enum('bike','car') not null,
    unique key uq_vehicles_plateNumber (plateNumber),
    key idx_vehicles_type (type)
);
```

```
create table Orders(
    id int primary key not null auto_increment,
    deliveryTime datetime,
    orderDate datetime not null,
    tenthOfGift boolean not null default false,
    clientId int not null,
    driverId int not null,
    vehicleId int not null,
    key idx_orders_clientId (clientId),
    key idx_orders_driverId (driverId),
    key idx_orders_vehicleId (vehicleId),
    key idx_orders_orderDate (orderDate),
    constraint fk_orders_client foreign key (clientId) references Clients(id) on delete
cascade,
    constraint fk_orders_driver foreign key (driverId) references Drivers(id) on delete
cascade,
    constraint fk_orders_vehicle foreign key (vehicleId) references Vehicles(id) on
delete cascade
);


create table OrderProduct(
    orderId int not null,
    productId int not null,
    quantity int not null,
    size enum('naine','humaine','ogresse') not null,
    primary key (orderId, productId),
    key idx_orderproduct_productId (productId),
    constraint chk_orderproduct_quantity_positive check (quantity >= 1),
    constraint fk_orderproduct_order foreign key (orderId) references Orders(id) on
delete cascade,
```

```
    constraint fk_orderproduct_product foreign key (productId) references Products(id)
on delete cascade
);
```

Script insertion des données dans la base

```
-- ========================
-- INGREDIENTS (PIZZA ONLY)
-- ========================
INSERT INTO Ingredients (name) VALUES
('Tomato sauce'),
('Mozzarella'),
('Ham'),
('Mushrooms'),
('Pepperoni'),
('Olives'),
('Onions'),
('Chicken'),
('Basil'),
('Parmesan');

-- ========================
-- PRODUCTS (PIZZAS ONLY)
-- ========================
INSERT INTO Products (name, basePrice) VALUES
('Margherita', 7.50),
('Pepperoni', 9.00),
('Hawaiian', 9.50),
('Chicken BBQ', 10.50),
('Four Cheese', 9.80);
```

```
-- ========================
-- PRODUCT_INGREDIENT
-- ========================

-- Margherita
INSERT INTO ProductIngredient VALUES
(1,1),(2,1),(9,1);

-- Pepperoni
INSERT INTO ProductIngredient VALUES
(1,2),(2,2),(5,2);

-- Hawaiian
INSERT INTO ProductIngredient VALUES
(1,3),(2,3),(3,3);

-- Chicken BBQ (approx BBQ vibe with chicken + onion + cheese base)
INSERT INTO ProductIngredient VALUES
(1,4),(2,4),(8,4),(7,4);

-- Four Cheese
INSERT INTO ProductIngredient VALUES
(1,5),(2,5),(10,5),(9,5),(6,5);
```

```
-- ========================
-- CLIENTS
-- ========================
INSERT INTO Clients (email, phoneNumber, firstName, lastName, address, password_hash) VALUES
('alice@example.com', '+33611111111', 'Alice', 'Martin', '10 Rue Paris', 'hash1'),
('bob@example.com', '+33622222222', 'Bob', 'Dupont', '20 Avenue Lyon', 'hash2'),
('carol@example.com', '+33633333333', 'Carol', 'Bernard', '5 Rue Lille', 'hash3');

-- ========================
-- DRIVERS
-- ========================
INSERT INTO Drivers (firstName, lastName, canBike, canDrive) VALUES
('David', 'Lopez', true, false),
('Emma', 'Durand', false, true),
('Lucas', 'Moreau', true, true);

-- ========================
-- VEHICLES
-- ========================
INSERT INTO Vehicles (plateNumber, brand, model, type) VALUES
('BIKE-001', 'Decathlon', 'Rockrider', 'bike'),
('CAR-001', 'Toyota', 'Yaris', 'car'),
('CAR-002', 'Renault', 'Clio', 'car');
```

```
-- ========================

-- ORDERS

-- ========================

INSERT INTO Orders (deliveryTime, orderDate, tenthOfGift, clientId, driverId,
vehicleId) VALUES

('2026-05-28 12:30:00', '2026-05-28 12:00:00', false, 1, 1, 1),

('2026-05-28 13:10:00', '2026-05-28 12:40:00', true, 2, 2, 2),

('2026-05-28 14:00:00', '2026-05-28 13:20:00', false, 3, 3, 3),

('2026-05-28 18:30:00', '2026-05-28 18:00:00', false, 1, 3, 2);


-- ========================

-- ORDER PRODUCT (PIZZAS ONLY)

-- ========================

INSERT INTO OrderProduct (orderId, productId, quantity) VALUES

(1, 1, 2), -- Margherita

(1, 2, 1), -- Pepperoni


(2, 3, 1), -- Hawaiian

(2, 4, 1), -- Chicken BBQ


(3, 2, 2), -- Pepperoni

(3, 5, 1), -- Four Cheese


(4, 1, 1), -- Margherita
```

(4, 4, 2); -- Chicken BBQ

Interrogation de la base de données

Menu

```
SELECT
    p.name AS Nom_Pizza,
    p.basePrice AS Prix,
    GROUP_CONCAT(i.name ORDER BY i.name ASC SEPARATOR ', ') AS Ingredients
FROM
    Products p
LEFT JOIN
    ProductIngredient pi ON p.id = pi.productId
LEFT JOIN
    Ingredients i ON pi.ingredientId = i.id
GROUP BY
    p.id,
    p.name,
    p.basePrice
ORDER BY
    p.name ASC;
```

Fiche de livraison

```
SELECT
    CONCAT(d.firstName, ' ', d.lastName) AS Nom_Livreur,
    v.type AS Type_Vehicule,
    CONCAT(c.firstName, ' ', c.lastName) AS Nom_Client,
    o.orderDate AS Date_Commande,
    TIMESTAMPDIFF(MINUTE, o.orderDate, o.deliveryTime) AS
Temps_Ecoule_Minutes,
    p.name AS Nom_Pizza,
    p.basePrice AS Prix_Base
FROM
    Orders o
JOIN
    Drivers d ON o.driverId = d.id
JOIN
    Vehicles v ON o.vehicleId = v.id
JOIN
    Clients c ON o.clientId = c.id
JOIN
    OrderProduct op ON o.id = op.orderId
JOIN
    Products p ON op.productId = p.id
ORDER BY
    o.orderDate DESC;
```

Questions diverses

Quels sont les véhicules qui n'ont jamais servi

```
SELECT * FROM Vehicles WHERE id NOT IN ( SELECT DISTINCT vehicleId FROM Orders);
```

Calculer le nombre de commandes par client

```
SELECT
    c.id AS Client_ID,
    CONCAT(c.firstName, ' ', c.lastName) AS Nom_Client,
    COUNT(o.id) AS Nombre_Commandes
FROM
    Clients c
LEFT JOIN
    Orders o ON c.id = o.clientId
GROUP BY
    c.id,
    c.firstName,
    c.lastName
ORDER BY
    Nombre_Commandes DESC,
    Nom_Client ASC;
```

Calcule de la moyenne des commandes

```
SELECT (SELECT COUNT(*) FROM Orders) / (SELECT COUNT(*) FROM Clients) AS Moyenne_Commandes_Par_Client;
```

Extraction des clients ayant commandé plus que la moyenne

```
SELECT
    c.id AS Client_ID,
    CONCAT(c.firstName, ' ', c.lastName) AS Nom_Client,
    COUNT(o.id) AS Nombre_Commandes
FROM
    Clients c
JOIN
    Orders o ON c.id = o.clientId
GROUP BY
    c.id,
    c.firstName,
    c.lastName
HAVING
    COUNT(o.id) > (SELECT COUNT(*) FROM Orders) / (SELECT COUNT(*) FROM Clients)
ORDER BY
    Nombre_Commandes DESC;
```

Suivi du chiffre d'affaires

```
SELECT
    ROUND(SUM(p.basePrice * op.quantity * CASE op.size
            WHEN 'naine' THEN 0.6667
            WHEN 'ogresse' THEN 1.3333
            ELSE 1.0000
        END
    ), 2) AS Chiffre_Affaires_Total
FROM
    OrderProduct op
JOIN
    Products p ON op.productId = p.id
JOIN
    Orders o ON op.orderId = o.id
WHERE
    o.tenthOfGift = FALSE
    AND (o.deliveryTime IS NULL OR TIMESTAMPDIFF(MINUTE, o.orderDate, o.deliveryTime) <= 30);
```

Identification du meilleur client

```sql
SELECT
    c.id AS Client_ID,
    CONCAT(c.firstName, ' ', c.lastName) AS Nom_Client,
    COUNT(o.id) AS Total_Commandes
FROM
    Clients c
JOIN
    Orders o ON c.id = o.clientId
GROUP BY
    c.id, c.firstName, c.lastName
ORDER BY
    Total_Commandes DESC
LIMIT 1;
```

Identification du plus mauvais livreur

```
SELECT
    d.id AS Livreur_ID,
    CONCAT(d.firstName, ' ', d.lastName) AS Nom_Livreur,
    v.plateNumber AS Vehicule_Utilise,
    COUNT(o.id) AS Nombre_Retards
FROM
    Drivers d
JOIN
    Orders o ON d.id = o.driverId
JOIN
    Vehicles v ON o.vehicleId = v.id
WHERE
    TIMESTAMPDIFF(MINUTE, o.orderDate, o.deliveryTime) > 30
GROUP BY
    d.id, d.firstName, d.lastName, v.plateNumber
ORDER BY
    Nombre_Retards DESC
LIMIT 1;
```

Identification de la pizza la plus demandée

```sql
SELECT
    p.name AS Nom_Pizza,
    SUM(op.quantity) AS Quantite_Totale_Vendue
FROM
    Products p
JOIN
    OrderProduct op ON p.id = op.productId
GROUP BY
    p.id, p.name
ORDER BY
    Quantite_Totale_Vendue DESC
LIMIT 1;
```

Identification de la pizza la moins demandée

```sql
SELECT
    p.name AS Nom_Pizza,
    SUM(op.quantity) AS Quantite_Totale_Vendue
FROM
    Products p
JOIN
    OrderProduct op ON p.id = op.productId
GROUP BY
    p.id, p.name
ORDER BY
    Quantite_Totale_Vendue ASC
LIMIT 1;
```

Identification de l'ingrédient favori

```sql
SELECT
    i.name AS Nom_Ingredient,
    SUM(op.quantity) AS Fois_Commande
FROM
    Ingredients i
JOIN
    ProductIngredient pi ON i.id = pi.ingredientId
JOIN
    OrderProduct op ON pi.productId = op.productId
GROUP BY
    i.id, i.name
ORDER BY
    Fois_Commande DESC
LIMIT 1;
```

Suppressions

```sql
DROP TABLE IF EXISTS OrderProduct;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS ProductIngredient;
DROP TABLE IF EXISTS Vehicles;
DROP TABLE IF EXISTS Drivers;
DROP TABLE IF EXISTS Clients;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Ingredients;
```

Procedure

```sql
DROP PROCEDURE IF EXISTS mark_order_delivered;

DELIMITER //

CREATE PROCEDURE mark_order_delivered(IN p_orderId INT)
BEGIN
    DECLARE v_orderDate DATETIME;
    DECLARE v_deliveryTime DATETIME;
    DECLARE v_clientId INT;
    DECLARE v_isGift BOOLEAN;
    DECLARE v_diff INT;
    DECLARE v_cost DECIMAL(10,2);
    DECLARE v_prod_price DECIMAL(5,2);
    DECLARE v_size VARCHAR(20);
    DECLARE v_quantity INT;
    DECLARE v_mult DECIMAL(5,4);

    -- Récupérer les infos commande
    SELECT orderDate, deliveryTime, clientId, tenthOfGift
    INTO v_orderDate, v_deliveryTime, v_clientId, v_isGift
    FROM Orders
    WHERE id = p_orderId;

    IF v_deliveryTime IS NULL THEN

        UPDATE Orders
        SET deliveryTime = NOW()
        WHERE id = p_orderId;

        SELECT deliveryTime
        INTO v_deliveryTime
        FROM Orders
        WHERE id = p_orderId;

        SET v_diff = TIMESTAMPDIFF(MINUTE, v_orderDate, v_deliveryTime);

        IF v_diff >= 30 AND v_isGift = FALSE THEN
```

```
        WHERE id = p_orderId;

        SET v_diff = TIMESTAMPDIFF(MINUTE, v_orderDate, v_deliveryTime);

        IF v_diff >= 30 AND v_isGift = FALSE THEN

            SELECT op.quantity, op.size, p.basePrice
            INTO v_quantity, v_size, v_prod_price
            FROM OrderProduct op
            JOIN Products p ON op.productId = p.id
            WHERE op.orderId = p_orderId
            LIMIT 1;

            IF v_size = 'naine' THEN
                SET v_mult = 0.6667;
            ELSEIF v_size = 'humaine' THEN
                SET v_mult = 1.0000;
            ELSEIF v_size = 'ogresse' THEN
                SET v_mult = 1.3333;
            END IF;

            SET v_cost = v_prod_price * v_mult * v_quantity;

            UPDATE Clients
            SET balance = balance + v_cost
            WHERE id = v_clientId;

        END IF;
    END IF;
END//

DELIMITER ;
```

Trigger

```sql
DROP TRIGGER IF EXISTS trg_orders_before_insert;
DROP TRIGGER IF EXISTS trg_orderproduct_after_insert;

DELIMITER //

-- Gestion de la fidélité : définir si c'est la 10ème commande
CREATE TRIGGER trg_orders_before_insert
BEFORE INSERT ON Orders
FOR EACH ROW
BEGIN
    DECLARE order_count INT;

    -- Compter combien de commandes le client a déjà passées
    SELECT COUNT(*) INTO order_count FROM Orders WHERE clientId = NEW.clientId;

    -- Si le client a déjà 9 commandes (ou un multiple de 9+1 etc), la 10ème est gratuite
    IF (order_count + 1) % 10 = 0 THEN
        SET NEW.tenthOfGift = TRUE;
    ELSE
        SET NEW.tenthOfGift = FALSE;
    END IF;
END //

-- Rembourser automatiquement si la commande nouvellement insérée est une commande gratuite (fidélité)
-- Étant donné que le système Java a déjà débité le solde du client avant l'insertion !
CREATE TRIGGER trg_orderproduct_after_insert
AFTER INSERT ON OrderProduct
FOR EACH ROW
BEGIN
    DECLARE is_gift BOOLEAN;
    DECLARE c_id INT;
    DECLARE order_cost DECIMAL(10,2);
    DECLARE prod_price DECIMAL(5,2);
    DECLARE mult DECIMAL(5,4);

    -- Vérifier si la commande correspondante est marquée comme un cadeau
    SELECT tenthOfGift, clientId INTO is_gift, c_id FROM Orders WHERE id = NEW.orderId;
```

```sql
CREATE TRIGGER trg_orderproduct_after_insert
AFTER INSERT ON OrderProduct
FOR EACH ROW
BEGIN
    DECLARE is_gift BOOLEAN;
    DECLARE c_id INT;
    DECLARE order_cost DECIMAL(10,2);
    DECLARE prod_price DECIMAL(5,2);
    DECLARE mult DECIMAL(5,4);

    -- Vérifier si la commande correspondante est marquée comme un cadeau
    SELECT tenthOfGift, clientId INTO is_gift, c_id FROM Orders WHERE id = NEW.orderId;

    IF is_gift THEN
        -- Retrouver le prix du produit
        SELECT basePrice INTO prod_price FROM Products WHERE id = NEW.productId;

        IF NEW.size = 'naine' THEN SET mult = 0.6667;
        ELSEIF NEW.size = 'humaine' THEN SET mult = 1.0000;
        ELSEIF NEW.size = 'ogresse' THEN SET mult = 1.3333;
        END IF;

        SET order_cost = prod_price * mult * NEW.quantity;

        -- Recréditer le solde du client car c'était la 10ème commande gratuite
        UPDATE Clients SET balance = balance + order_cost WHERE id = c_id;
    END IF;
END //

DELIMITER ;
```

Programmation et Architecture de l'Application

L'application de gestion est développée en Java. Nous avons structuré le code de manière modulaire. Le code est divisé en trois packages principaux. Cela sépare l'interface visuelle de la base de données.

Le Modèle de Données (Package model) Ce package contient les objets métiers. Chaque grande entité de la base de données possède sa propre classe Java. Nous avons créé les classes `Client`, `Product`, `Order`, `DeliveryGuy` et `Vehicle`. Ces classes sont simples. Elles possèdent des attributs privés et des méthodes d'accès (getters et setters). Elles stockent temporairement les données en mémoire.

L'Accès aux Données (Package dao) Nous avons mis en place le patron de conception DAO (Data Access Object). Ce choix technique centralise et isole toutes les requêtes SQL. L'application communique avec la base via l'API JDBC. La connexion est gérée par la classe `DatabaseConnection`. Cette classe utilise le pattern Singleton pour garantir une connexion unique. Elle lit les identifiants dans un fichier `db.properties`. Cela évite de laisser les mots de passe en clair dans le code Java. Les classes DAO utilisent des objets `PreparedStatement` pour sécuriser les requêtes. Nous utilisons également un `CallableStatement` dans la classe `OrderDao`. Cela permet de déclencher notre procédure stockée `mark_order_delivered` directement depuis l'interface.

L'Interface Graphique (Package views) L'interface utilisateur est construite avec l'API Java Swing. La navigation commence par la classe `MainWindow`. Elle affiche un menu principal avec plusieurs boutons. Chaque action ouvre une nouvelle fenêtre (`JFrame`). Nous exploitons largement le composant `JTable`. Ce composant est très pratique pour afficher des listes. Nous l'utilisons pour la carte des pizzas, les fiches de livraison et le classement des livreurs. L'affichage est géré dynamiquement par la classe `DefaultTableModel`. Elle fait le lien parfait entre les listes générées par nos DAO et l'interface visuelle.

Arborescence